# Ember Attractor — Performance Report

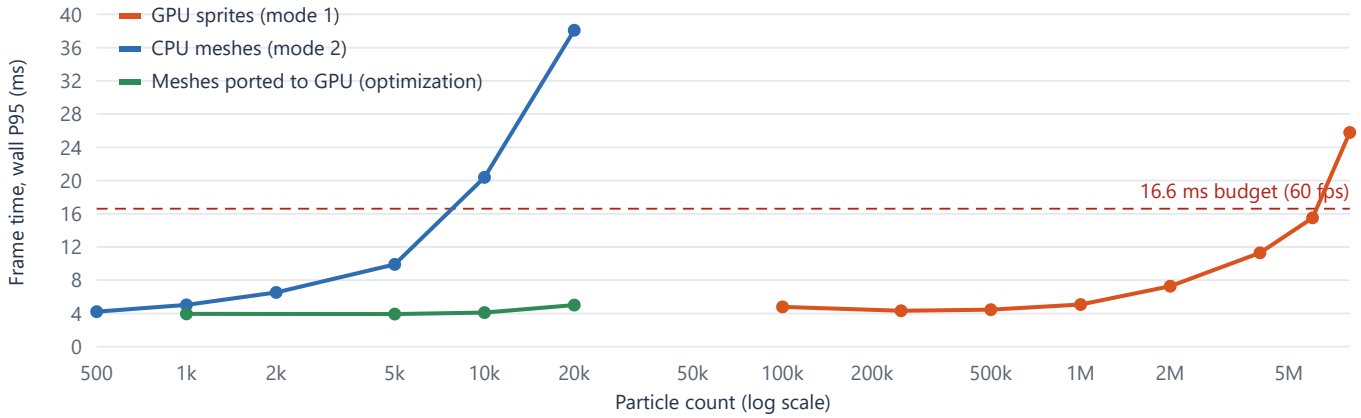
Niagara at Scale · UE 5.7.4 · Intel Core Ultra 7 265F, GeForce RTX 5070 Ti, 32 GB, Windows 11, D3D12, 1920×1080, VSync off

## Budget and method

**Budget (set before building):** 60 fps, i.e. 16.6 ms per frame at 1080p, with Niagara at most 4 ms of combined game- and render-thread time. GPU time is reported separately.

**Method:** each count level is a fresh standalone game (Development editor build, `-game`, offscreen rendering) in the lab sandbox (`ember.Game 0`). That keeps the particle population constant while an automated attractor orbits and pulls. Each run has a 6 s warm-up and a 10 s capture, with `stat Niagara`, `stat GPU` and the Niagara Debugger HUD on. Frame time is wall-clock P95 from the game's own CSV, excluding three flagged screenshot frames. Niagara GT+RT is `Particles/Total` + `Particles/RT` per system from the engine CSV (`fx.DetailedCSVStats`). Unreal Insights traces were recorded separately for the default game and both breaking points. The stat overlays themselves cost about 1.1 ms of game-thread time (`DebugHUD`) in every run. Numbers come from a strong workstation, not the lab machines.

## Scaling: frame time against particle count




**System 1 — GPU sprites (mode 1)**

| Count | Wall mean | Wall P95 | GPU frame | Niagara GT+RT | Niagara GPU sim | Budget |
|---|---|---|---|---|---|---|
| 100k | 3.79 | 4.78 | 2.11 | 0.135 | 0.055 | pass |
| 250k | 3.58 | 4.32 | 2.40 | 0.130 | 0.086 | pass |
| 500k | 3.76 | 4.46 | 2.95 | 0.133 | 0.179 | pass |
| 1M | 4.51 | 5.06 | 3.97 | 0.123 | 0.363 | pass |
| 2M | 6.70 | 7.29 | 6.11 | 0.129 | 0.753 | pass |
| 4M | 10.73 | 11.29 | 10.19 | 0.129 | 1.457 | pass |
| 6M | 14.89 | 15.51 | 14.35 | 0.128 | 2.179 | pass |
| **8M** | **24.91** | **25.77** | **24.33** | **0.139** | **2.908** | **fail** |

**System 2 — CPU meshes (mode 2)**

| Count | Wall mean | Wall P95 | GPU frame | Niagara GT+RT | Niagara GPU sim | Budget |
|---|---|---|---|---|---|---|
| 0.5k | 3.39 | 4.21 | 1.70 | 0.460 | — | pass |
| 1k | 4.17 | 5.04 | 1.73 | 0.744 | — | pass |
| 2k | 5.50 | 6.54 | 1.75 | 1.340 | — | pass |
| 5k | 8.67 | 9.91 | 1.77 | 2.847 | — | pass |
| **10k** | **16.50** | **20.37** | **1.90** | **5.265** | **—** | **fail** |
| **20k** | **31.57** | **38.10** | **2.09** | **10.225** | **—** | **fail** |

All values in ms. Baseline with no particles: wall mean 2.43, GPU frame 1.56. Red rows miss the budget (wall P95 > 16.6 ms or Niagara GT+RT > 4 ms).

## Where each system breaks

**GPU sprites break on rendering, not on Niagara.** Niagara's CPU-side cost stays flat at 0.12–0.14 ms from 100k to 8M particles: the CPU only dispatches the simulation. The GPU simulation itself scales linearly at about 0.36 ms per million particles (2.9 ms at 8M). The rest of the GPU frame is translucent sprite rendering, which grows from 2.1 ms at 100k to 14.4 ms at 6M and 24.3 ms at 8M. 6M still passes (P95 15.5 ms) and 8M fails (P95 25.8 ms), so the budget breaks between 6M and 8M. The render thread is waiting on the GPU, which makes this a fill-rate and overdraw limit.

**CPU meshes break on the game thread.** Niagara's own stat grows by about 0.5 µs per particle (0.46 ms at 500, 2.85 ms at 5k, 5.27 ms at 10k), so the 4 ms Niagara budget is crossed at roughly 7k particles. The whole-frame budget fails at 10k (P95 20.4 ms). The engine CSV shows why the frame cost rises faster than the Niagara counter: the collision module's scene traces cost 7.1 ms (`CoreSystems_Collision`) plus 4.2 ms of async-trace bookkeeping at 10k. Most of that is outside the per-system Niagara stat.

## Optimization: fixed bounds on the CPU system

**Before → after: fixed bounds.** The CPU system normally computes its bounds every frame. The shipped assets use fixed bounds that cover the chamber. `ember.CPUDynamicBounds 1` (asset `NS_Debris_CPU_DynamicBounds`) restores the per-frame calculation so the two can be compared on an identical workload:

| CPU meshes | Dynamic bounds (before) | Fixed bounds (after) | Change | Wall mean |
|---|---|---|---|---|
| 5,000 | 2.837 | 2.847 | +0.3% | 9.19 → 8.67 |
| 20,000 | 10.526 | 10.225 | -2.9% | 33.76 → 31.57 |

The change in Niagara cost is within run-to-run noise (+0.3% at 5k, −2.9% at 20k), even though wall time dropped 6–7%. The honest conclusion: bounds calculation is not where the time goes for 20k meshes. The profile above points at collision traces instead.

**Before → after: CPU-to-GPU port.** The same debris emitter (meshes, the same forces, export and kill) simulated on the GPU, selected with `ember.DebrisOnGPU 1`. At 20k meshes, Niagara GT+RT falls from 10.2 ms to 0.15 ms and frame P95 from 38.1 ms to 5.0 ms. At 10k it falls from 5.27 to 0.13 ms (P95 20.4 → 4.1 ms), while the GPU simulation costs only 0.03 ms. The price: GPU collision cannot trace arbitrary scene geometry the way the CPU can. Scene-depth collision killed the whole population in testing, so the port collides analytically with the floor plane. That is correct for this flat chamber and wrong for any other level. Gameplay data also arrives a few frames later through readback, and per-particle state is no longer cheaply readable on the CPU.

| Meshes | CPU sim GT+RT | GPU port GT+RT | CPU sim P95 | GPU port P95 | GPU port sim |
|---|---|---|---|---|---|
| 1,000 | 0.744 | 0.131 | 5.04 | 3.95 | 0.032 |
| 5,000 | 2.847 | 0.129 | 9.91 | 3.92 | 0.032 |
| 10,000 | 5.265 | 0.133 | 20.37 | 4.11 | 0.033 |
| 20,000 | 10.225 | 0.149 | 38.10 | 5.00 | 0.036 |

## Many simple vs few complex

At about the same 10 ms frame, the GPU system draws 4,000,000 sprites (10.7 ms mean) while the CPU system manages 5,000 meshes (8.7 ms mean): roughly 800× more particles. Per particle the CPU path costs about 0.5 µs of game-thread time (plus scene traces); the GPU path costs about 0.36 ns of GPU simulation plus rendering. For a lead engineer: use GPU simulation whenever gameplay only needs an *aggregate* answer that can arrive a few frames late (how much matter crossed the horizon). Keep CPU simulation for the few particles that need exact same-frame state, arbitrary-geometry collision, or per-particle events that drive logic. This game does both: the dust is the GPU many, the debris the CPU few, and both feed the same readback path.

## Gameplay readback

Getting data off the GPU is solved with an event-driven export, not a full readback. A particle is written to the *Export Particle Data* interface only in the frame it crosses the event horizon, and a *Kill Particles* module reads that same export flag, so each particle is reported exactly once. A 100,000 + 500 test round reported exactly 100,000 GPU and 500 CPU particles. The GPU reserves 16,384 entries per frame. A 4M-particle round swallowed 2.04M particles, at a peak of about 210,000 particles per second, without ever filling that reservation. The batch carries each particle's age, which becomes heat and then mass. Mass sets the horizon radius, pull strength, pull radius and vortex, which go back into Niagara as user parameters: a closed two-way loop. GPU readback latency (a few frames) was not measured precisely.

## With another week

Measure readback latency per frame in Insights and run the sweep on the lab machines. Cut sprite overdraw, which is the GPU limit: smaller or opaque sprites, plus an Effect Type with distance and budget scaling. Fix scene-depth collision for the GPU port so it works beyond a flat floor. Give the debris neighbor-grid behaviour (clumping into larger chunks) to make the few-complex system matter more in play. Then play-test the energy and cooling numbers with other people.

Evidence: Evidence/Final (CSVs, logs, SHA-256 manifest), Documentation/Sweep.md. Reproduce with Tools/Sweep.ps1 then Tools/SummarizeSweep.py and Tools/MakeReport.py.